The endpoint filter pipeline is similar to the middleware pipeline in many ways, but you should consider several subtle differences when deciding which approach to use. The similarities include three main parallels:

- *Requests pass through a middleware component on the way in, and responses pass through again on the way out.* Similarly, endpoint filters can run code before calling the next filter in the pipeline and can run code after the response is generated, as shown in figure 5.8.
- *Middleware can short-circuit a* request *by returning a response instead of passing it on to later middleware.* Filters can also short-circuit the filter pipeline by returning a response.
- *Middleware is often used for cross-cutting application concerns, such as logging, performance profiling, and exception handling.* Filters also lend themselves to cross-cutting concerns.

By contrast, there are three main differences between middleware and filters:

- Middleware can run for all requests; filters will run only for requests that reach the `EndpointMiddleware` and execute the associated endpoint.
- Filters have access to additional details about the endpoint that will execute, such as the return value of the endpoint, for example an `IResult`. Middleware in general won't see these intermediate steps, so it sees only the generated response.
- Filters can easily be restricted to a subset of requests, such as a single endpoint or a group of endpoints. Middleware generally applies to all requests (though you can achieve something similar with custom middleware components).